Architectural Drivers

ARCHITECTURALLY SIGNIFICANT REQUIREMENTS

Functional vs. Non-Functional Requirements

NOT CALLED NON-FUNCTIONAL ANYMORE

Technical vs. Business Constraints

Business: "We must launch by Black Friday," "Budget is $5k/month.“

Technical: "Must integrate with 20-year-old COBOL mainframe," "Team only knows Python."

The Three Pillars of a Request

1. Functional: What it does (User can login).
2. Non-Functional (Quality Attributes): How it behaves (Login takes < 2s).
3. Constraints: What limits us (Must use AWS).

The “-ilities” (Quality Attributes)

Availability: Is the system up?

Scalability: Can it handle more users?

Maintainability: Can we fix it easily?

Security: Is the data safe?

Performance vs. Scalability

Performance: A single user's experience (Latency).

Scalability: The system's capacity for 1,000,000 users (Throughput).

Quality Attributes

99.9%

Reliability

Percentage indicating the dependable uptime and correct functioning of systems.

10K+

Transactions

Signifies the scale of operations a system can handle, testing the limits of performance and throughput.

0.1s

Latency

Threshold for response times, indicating swift and efficient system reactions.

Extracting characteristics from requirements.

Some characteristics come from explicit statements in requirements documents.

For example, expected number of users and scale commonly appear explicitly.

Extracting characteristics from domain concerns.

An architect must be able to translate domain concerns into characteristics.

Is scalability the biggest concern? Or is it performance? Perhaps security?

When collaborating with domain stakeholders try to make the list as short as possible.

Each characteristic supported, complicates the design.

The number of characteristics is less important than the simplicity of the design.

Extracting characteristics from domain concerns.

A hint is to make the stakeholders select the top three characteristics.

This can lead to a discussion among all parties on what is important.

The issue comes with wording and terms translations: architects and stakeholders may have different terms for each concept.

For the most frequent concerns we can select a number of characteristics that may apply to them.

Extracting characteristics from domain concerns.

Domain Concern.	Architecture Characteristics.
Mergers and Acquisitions.	Interoperability, scalability, adaptability, extensibility.
Time to market.	Agility, testability, deployability.
User satisfaction,	Performance, availability, fault tolerance, testability, deployability, agility, security.
Competitive Advantage.	Agility, testability, deployability, scalability, availability, fault tolerance.
Time and budget.	Simplicity, feasibility.

Taxonomy

ISO 25010 Quick Reference Guide

Quality Characteristic	Brief Description
Functional Suitability	The degree to which a product provides functions that meet stated and implied needs, including functional completeness, correctness, and appropriateness.
Performance Efficiency	The degree to which a system performs within specified time and throughput parameters while remaining efficient in its use of resources like CPU and memory.
Compatibility	The ability of a product to exchange information with other systems or share a common environment and resources without detrimental impact (co-existence).
Interaction Capability	The degree to which a product can be used by specific users to complete tasks; this encompasses usability, learnability, and user engagement.
Reliability	The degree to which a system performs specified functions under stated conditions for a period of time, including its ability to recover from crashes.

ISO 25010 Quick Reference Guide

Quality Characteristic	Brief Description
Security	The degree to which a product protects information and data so that persons or other systems have the appropriate level of data access and authorization.
Maintainability	The effectiveness and efficiency with which a system can be modified by developers to fix bugs, improve performance, or add new features.
Flexibility	The degree of efficiency with which a product can be adapted for different hardware, software, or other operational environments (includes scalability and adaptability).
Safety	The degree to which a system limits the risk of harm to people, business, or the environment, focusing on operational constraints and fail-safe integration

Quality attributes: ISO 25010

SOFTWARE PRODUCT QUALITY								
FUNCTIONAL SUITABILITY	PERFORMANCE EFFICIENCY	COMPATIBILITY	INTERACTION CAPABILITY	RELIABILITY	SECURITY	MAINTAINABILITY	FLEXIBILITY	SAFETY
FUNCTIONAL COMPLETENESS FUNCTIONAL CORRECTNESS FUNCTIONAL APPROPRIATENESS iso25000.com	TIME BEHAVIOUR RESOURCE UTILIZATION CAPACITY	CO-EXISTENCE INTEROPERABILITY	APPROPRIATENESS RECOGNIZABILITY LEARNABILITY OPERABILITY USER ERROR PROTECTION USER ENGAGEMENT INCLUSIVITY USER ASSISTANCE SELF-DESCRIPTIVENESS	FAULTLESSNESS AVAILABILITY FAULT TOLERANCE RECOVERABILITY	CONFIDENTIALITY INTEGRITY NON-REPUDIATION ACCOUNTABILITY AUTHENTICITY RESISTANCE	MODULARITY REUSABILITY ANALYSABILITY MODIFIABILITY TESTABILITY	ADAPTABILITY SCALABILITY INSTALLABILITY REPLACEABILITY	OPERATIONAL CONSTRAINT RISK IDENTIFICATION FAIL SAFE HAZARD WARNING SAFE INTEGRATION

Operational Architectural characteristics

TERM	DEFINITION
Availability	How long the system needs to be available.
Continuity	Disaster recovery capacity.
Performance	Includes stress testing, peak analysis, capacity required and response times.
Recoverability	Business continuity requirements. Backup strategies and duplicated hw requirements.
Reliability/Safety	Assess if the system needs to be fail-safe. What happens if it fails?
Robustness	Ability to handle error and boundary conditions. For instance: network or hw issues, power outages, etc.
Scalability	Ability for the system to perform and operate as the number of users or requests increases (along time).

Structural Architectural characteristics

TERM	DEFINITION
Configurability	Ability for end-users to easily change aspects of the software configuration.
Extensibility	How important is to plug new pieces of functionality in.
Installability	Ease of system installation and it’s platforms.
Reuse	Ability to leverage common components across multiple products.
Localization	Support for multiple languages, units of measures or currencies.
Maintainability	How easy is it to apply changes to enhance the system.
Portability	Does the system need to run on more than one platform?
Supportability	What level of technical support is needed by the application?
Upgradeability	Ability to easily/quickly upgrade from a previous version to a newer version.

Cross-cutting Architectural characteristics

TERM	DEFINITION
Accessibility	Access to all your users, including those with any type of disability.
Archivability	Will the data be archived or deleted after a period?
Authentication	Security requirements to ensure users are who they say they are.
Authorization	Security requirements to ensure users can access only certain functions within the application.
Legal	What legislative constraints is the system operating in. What reservation rights does the company require? Regulations on building or deploying the application.
Privacy	Ability to hide transactions from internal company employees.
Security	Does the data need to be encrypted in the database? For network communications? Authentication for remote access?
Usability	Level of training required by users to achieve their goals with the application.

Comparison of Overlapping Characteristics

Characteristic	Focus	Key Question
Reliability	**Continuity**	How long can the system run without a failure?
Availability	**Readiness**	Is the system up and running right now when I need it?

Characteristic	Focus	Key Question
Interoperability	**Interaction**	Can two *different* systems exchange and use information?
Compatibility	**Coexistence**	Can two components perform their functions in the same environment?

Disclaimers

Any given list of characteristics will be incomplete.

Many of the terms are ambiguous and imprecise (for instance: interoperability and compatibility).

Many definitions overlap, such as availability and reliability.

The International Organization for Standardization (ISO) publishes a list of characteristics by capabilities.

Homework: Search the official ISO list and describe at least two that your system may have and why you consider them so.

Case study

Case study: Silicon sandwiches.

Description:

- A national sandwich shop wants to enable online ordering (in addition to its current call-in service).

Users:

- Thousands, perhaps one day, millions.

Requirements:

- Users will place their order, then be given a time to pick up and directions to the shop.
- If the shop offers delivery, dispatch the driver with the order to the user.
- Mobile-device accessibility.
- Offer national daily promotions/specials.
- Offer local daily promotions/specials.
- Accept payment online, in person or upon delivery.

Case study: Silicon sandwiches.

Additional context:

- Sandwich shops are franchised, each with a different owner.
- Parent company has near-future plans to expand overseas.
- Corporate goal is to hire inexpensive labor to maximize profit.

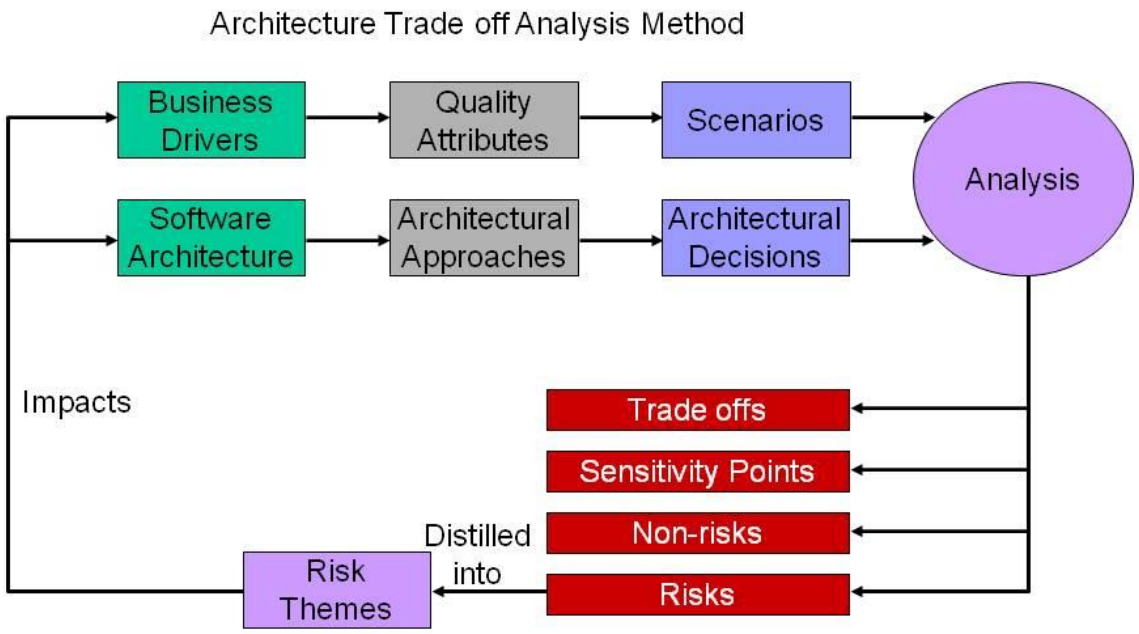
Quantifying and Prioritizing

ARCHITECTURE TRADE-OFF ANALYSYS METHOD

ATAM

ARCHITECTURE TRADE-OFF ANALYSIS METHOD

Architecture Trade-off Analysis Method



Steps & Outcomes

STEPS

Present the ATAM.

Present business drivers.

Present architecture.

Identify architectural approaches.

Generate quality attribute utility tree.

Analyze architectural approaches.

Brainstorm and prioritize scenarios.

Analyze architectural approaches.

Present results.

OUTCOMES

A concise presentation of the architecture.

Articulation of business goals.

The quality requirement in terms of a collection of scenarios.

Mapping of architectural decisions to quality requirements.

A set of identified sensitivity and tradeoff points.

A set of risks and non-risks.

A set of risk themes.

Quality Attribute Scenarios

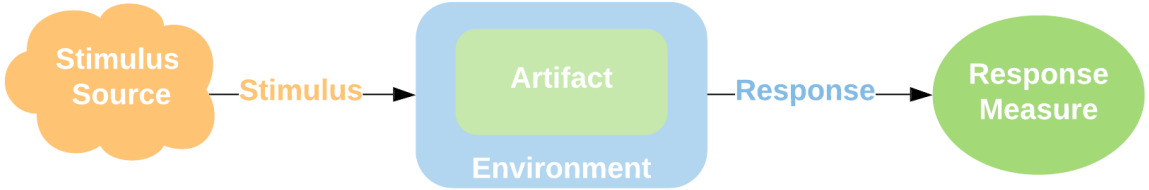
Why Scenarios?

Problem: "The system must be fast" is subjective and untestable.

Solution: Scenarios provide measurable, objective benchmarks

The 6-Part Anatomy

Part	Description	Example (Availability Scenario)
1. Source	Who or what generated the event?	An external monitoring service.
2. Stimulus	What is the actual event/condition?	The Primary Database instance crashes.
3. Artifact	What part of the system is affected?	The Persistence Layer / Database Cluster.
4. Environment	What is the system's current state?	During peak traffic hours (Normal Ops).
5. Response	How should the system react?	System switches to a "Read-Only" replica.
6. Measure	The measurable success criteria.	Switchover completes in **< 30 seconds**.



Silicon Sandwiches

Quality Attribute	Driving Business Requirement / Statement	1. Source	2. Stimulus	3. Artifact	4. Environment	5. Response	6. Measure
Scalability	"Users: Thousands, perhaps one day, millions."	Thousands of customers	Concurrent order volume increases to 1,000,000 users	Infrastructure / Order Processing System	Peak promotion period	System auto-scales resources to handle the throughput	All transactions processed without dropping orders

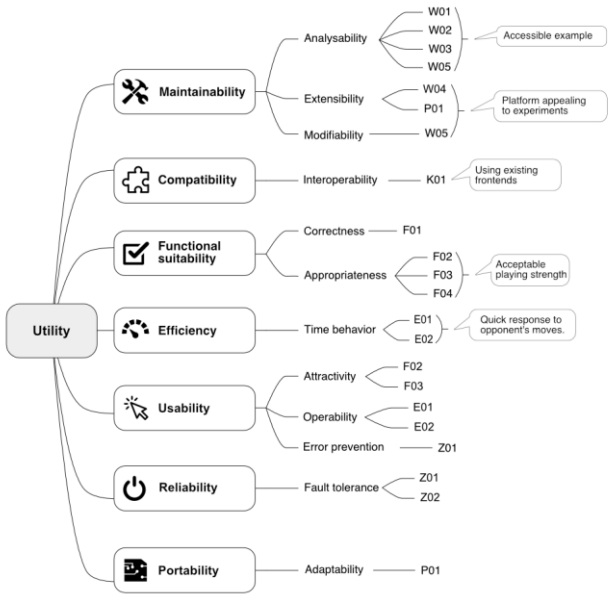
Prioritization & Utility Trees

What is a Utility Tree?

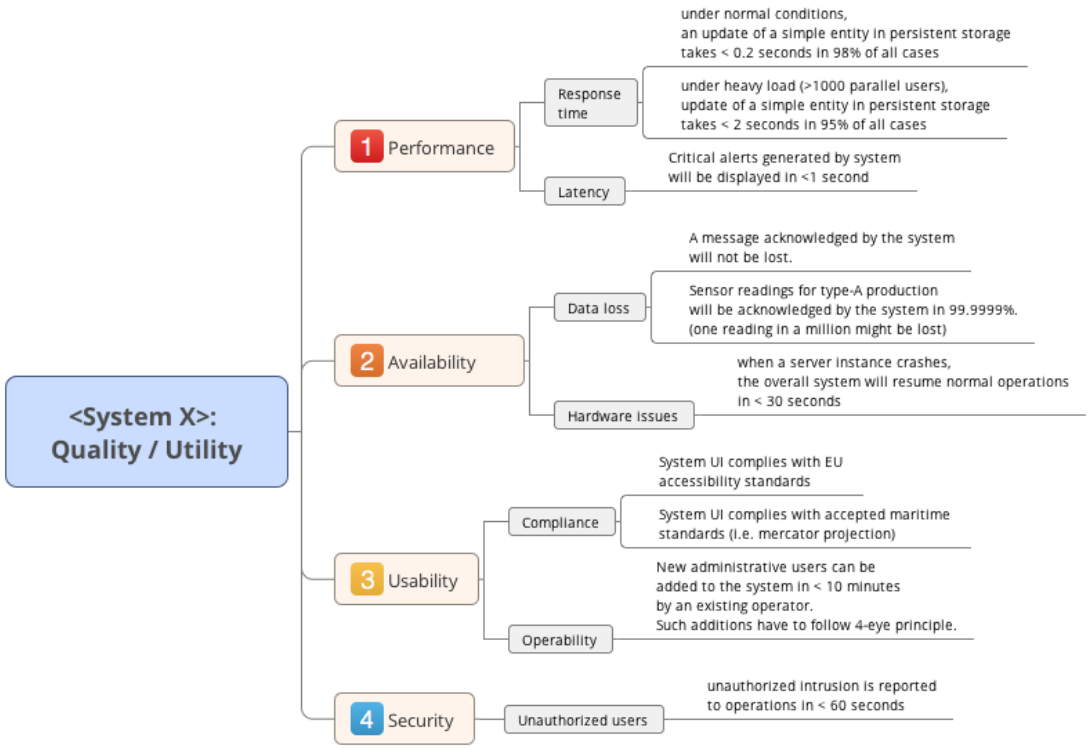
Definition: A top-down tool to translate broad business "Utility" into specific, measurable scenarios.

Structure:

- **Root:** Utility (The overall value of the system).
- **Branches:**
  - **Quality Attributes:** (e.g., Performance, Modifiability).
  - **Refinement:** (e.g., Data Latency, Ease of adding new payment methods).
  - **Scenario:** The 6-part scenario we built in Deck 5.



Utility tree example



Refinement and scenarios

Quality Attributes: (e.g., Performance, Modifiability).

Refinement: (e.g., Data Latency, Ease of adding new payment methods).

Scenario: The 6-part scenario we built in Deck 5.

Quality Attribute	Refinement	Scenario (The Leaf)
Availability	Server Failure	Database fails; system stays in "Read Only" mode (< 1 min).
Performance	Transaction Speed	Checkout completes in < 2 seconds under 5k concurrent users.
Modifiability	New Features	Adding a new shipping provider takes < 10 person-days.

Utility Tree (Silicon Sandwiches)

Quality Attribute	Refinement	Scenario (The Leaf)
Scalability	High Throughput	Concurrent order volume increases to 1,000,000 users during a national promotion; system auto-scales and processes all transactions without dropping orders.

The ASR Prioritization Matrix

Concept: Every leaf of the Utility Tree must be ranked.

The Two Dimensions:

- Business Value: How important is this to the stakeholder?
- Architectural Risk/Impact: How hard is this to implement or change later?

	Low Architectural Impact	High Architectural Impact
High Business Value	**"Quick Wins" (Prioritize Second)**	**"Architectural Drivers" (Top Priority)**
	Example: New User Profile Page UI	Example: Real-time fraud detection engine
Low Business Value	**"Fillers/Nice to Have"**	**"Technical Risks" (Prioritize Third)**
	Example: Color theme switcher	Example: Supporting legacy 32-bit OS

ASR Priorization Matrix (Silicon Sandwiches)

Scenario (Architectural Characteristic)	Business Value	Architectural Impact	Priority	Rationale
Scalability: Handle 1,000,000 concurrent users during promotions.	**High**	**High**	**1**	**Architectural Driver.** Defines the core infrastructure, database sharding, and caching strategy.

Trade-offs

THE REALITY

The Architect's Dilemma

The Concept: You cannot optimize for everything. An architect is a manager of Scarcity (Time, Budget, Performance).Bullets:

The Conflict:

- High Security often reduces Performance (encryption overhead).
- High Availability increases Cost (redundant servers).

The Solution: Prioritization based on Architecturally Significant Requirements (ASRs).

The Architecture Trade-off Matrix

Goal	Usually Conflicts With...	Why?
High Security	**Performance**	Encryption and multi-factor checks add latency and processing overhead.
High Availability	**Cost / Complexity**	Requires redundant servers, data replication, and failover mechanisms.
High Flexibility	**Maintainability**	"Generic" solutions (as you noted) lead to complex code that is hard to debug.
High Performance	**Portability**	Optimization often requires hardware-specific code that won't run everywhere.

Trade-offs and least bad architecture

Applications can only support a few of the mentioned characteristics.

- Each supported one requires design effort and structural support.
- Each characteristic impacts others; for instance, security and performance.

Too many characteristics lead to generic solutions that rarely work because design becomes unwieldy.

Architects should strive to design architecture to be as iterative as possible.

Agile software development is about iteration.

"Never shoot for the best architecture, but rather the least bad architecture."

Trade-off Identification

Concept: Once you have your ASRs, you find the “Tension Points”.

Example Case:

- **ASR 1:** "Extreme Security" (Encryption at rest and in transit).
- **ASR 2:** "Sub-millisecond Latency.“
- **The Trade-off:** The architect must document that ASR 1 will negatively impact ASR 2, and the stakeholder must sign off on which one "wins."

Trade-off analysis (Silicon Sandwiches)

Tension Point	Goal A (High Priority)	Goal B (Conflicting Factor)	Why? (The Architectural Trade-off)
Scalability vs. Simplicity	**Scalability** (Handling millions of users)	**Simplicity / Time to Market** (Black Friday launch or budget)	Designing for extreme scale (throughput) requires complex infrastructure like **database sharding and caching**, which conflicts with the need for a simple, fast-to-deploy design.

Conclusions.

There is not a right answer for the set of characteristics.

We must see which ones are key to success or must be enforced due to domain-concerns.

A useful exercise is to make another pass to the list of characteristics/requirements to see if we missed something.

ADRs

ARCHITECTURE DECISION RECORDS

The "Why did we do this?" Problem

The Scenario: You join a project. The database is a graph database (Neo4j), but the data is just simple user profiles. You ask "Why?". The only answer is "Dave thought it was cool in 2022." Dave left the company in 2023.

The Cost: Technical debt, confusion, and fear of changing anything.

The Solution: Architecture Decision Records (ADRs).

What is an ADR?

A short text file (usually Markdown) that captures a significant architectural decision.

It's **immutable history**: You don't edit an old ADR; you create a new one that supersedes it.

Stored in Git: Right next to the code it describes.

When do you write one?

Not for every function or variable name.

The "Significant" Rule: If the decision is hard to change later, affects multiple teams, or involves a trade-off between Quality Attributes (e.g., Performance vs. Cost).

The Anatomy of a Decision

Title: Short and descriptive.

Status: Proposed, Accepted, Superseded, Deprecated.

Context: What was the problem? What were the constraints? (The "Why").

Decision: What are we doing? (The "What").

Consequences: What is the "price" we pay? (The "Trade-off").

Benefits for the Team

Onboarding: New hires read the ADR folder to understand the system's "soul."

Alignment: No more circular arguments in meetings. "We already decided this in ADR-005."

Accountability: Ownership of the choice and its fallout.

ADR Template

ADR-00X: [Short, Descriptive Title]

Date: YYYY-MM-DD

Status: [Proposed | Accepted | Superseded by ADR-00Y]

Consulted: [Team Members / Stakeholders involved]

1. Context

Describe the technical/business problem we are facing. Include constraints (budget, time, existing tech stack). *Example: "We need to store 100k messages per second. Our current relational DB is locking up."*

2. Decision

State the choice clearly. Use active voice. *Example: "We will migrate the messaging module to Apache Cassandra."*

3. Alternatives Considered

Briefly list what else was looked at and why it wasn't chosen.

- *Redis (Too volatile for long-term storage)*
- *PostgreSQL Sharding (Too much operational overhead)*

4. Consequences

Positive (The Gains): What gets better? (e.g., High availability, horizontal scaling).

Negative (The Pains): What do we lose? (e.g., No more ACID transactions, team needs to learn CQL).